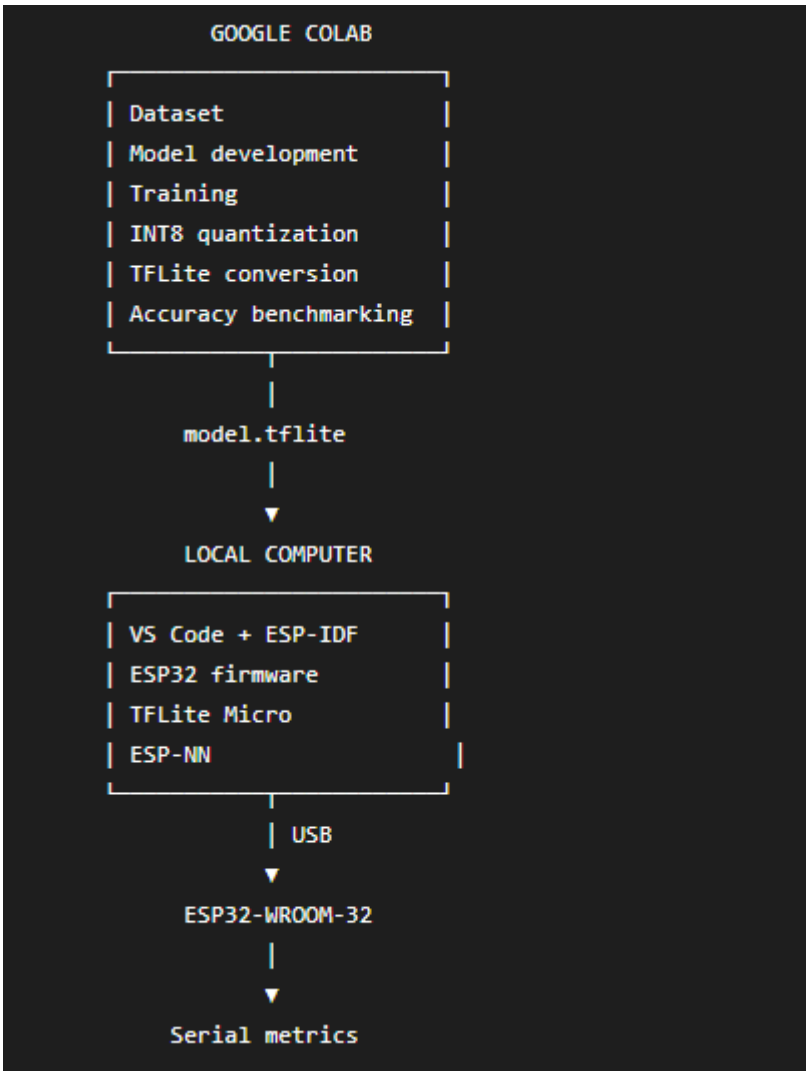
```
================================
ESP32 HARDWARE INFORMATION
================================
Chip model       : ESP32-D0WD-V3
CPU frequency    : 240 MHz
Flash size       : 4194304 bytes
Free heap        : 335332 bytes
Minimum free heap: 329704 bytes
Sketch size      : 297536 bytes
Free sketch space: 1310720 bytes
================================
```

For the actual deployment, we'll use **TFLite Micro + ESP-NN through ESP-IDF**. Espressif's current `esp-tflite-micro` component is designed for ESP-IDF and supports ESP-IDF 5.1–6.0; it also integrates ESP-NN optimized kernels.



ESP32 + TinyML Deployment Experiment — Progress Report

1. Objective

The original objective of this experiment was to answer a very specific question:

> **Can a small neural-network model be converted to a format suitable for embedded inference and actually run on the physical ESP32?**

The intention was **not yet** to deploy the washing-machine command model or the existing SmolLM2-135M model.

Instead, we deliberately started with a **tiny neural network** so that we could isolate the embedded-deployment problem:

Train tiny model
↓
Convert to TFLite
↓
Quantize to INT8
↓
Verify locally
↓
Embed model into ESP32 firmware
↓
Compile with ESP-IDF
↓
Flash physical ESP32
↓
Initialize TFLite Micro
↓
Run inference
↓
Measure latency / memory / flash

We have currently reached the **TFLite Micro model initialization stage on real hardware**.

---

2. Hardware Used

The physical target is the original ESP32 board:

Board

DOIT ESP32 DEVKIT V1

Detected chip

ESP32-D0WD-V3
Revision: v3.1

Hardware capabilities reported by ESP-IDF

WiFi
Bluetooth
Dual Core
240 MHz hardware

Flash

The board was detected as:

4 MB SPI Flash

We also explicitly configured ESP-IDF for:

Flash size: 4 MB
Flash mode: DIO
Flash frequency: 40 MHz

The serial port used was:

COM6

---

3. Why We Did Not Start With SmolLM2

The existing project involves **SmolLM2-135M-Instruct**, but putting a 135M-parameter language model directly onto an ordinary ESP32-D0WD-V3 would be a completely different problem.

The ESP32 has limited:

- SRAM
- flash
- compute
- memory bandwidth

Therefore, starting with the actual SLM would make it difficult to distinguish:

"The model is too large"

from:

"The embedded deployment pipeline isn't working."

So we intentionally created a **tiny neural network** first.

This gives us a clean compatibility baseline.

---

4. Colab Environment Setup

We chose **Google Colab** for the model/conversion side.

The reason was practical: model experimentation and conversion are easier in Colab, avoiding local GPU/driver issues.

The environment successfully reported:

Python 3.13.15
TensorFlow 2.20.0

We also verified that:

- TensorFlow was available
- TFLite conversion was available
- TFLite Interpreter was available

We created the working directory:

/content/esp32_washing_machine/

with:

models/
data/
exports/
benchmarks/

At this stage, **no washing-machine dataset was required**.

---

5. Tiny Neural Network Created

We created a very small Keras model:

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1)
])
```

Its purpose was not to solve a meaningful application problem.

It was simply a **deployment test model**.

The model was trained to approximate:

$y = \sin(x)$

using 1,000 training points.

Training configuration included:

Epochs: 50
Batch size: 32

---

6. TFLite Conversion

We converted the trained Keras model into TensorFlow Lite.

Two versions were generated.

Float32 model

tiny_sine_float32.tflite

Size:

3164 bytes

INT8 model

tiny_sine_int8.tflite

Size:

3344 bytes

Interestingly, the INT8 model was slightly larger:

Float32: 3164 B

INT8: 3344 B

That is not an error.

Quantization can introduce additional metadata/tensor quantization information, so **INT8 does not guarantee a smaller `.tflite` file for a tiny model**.

---

7. INT8 Model Verification in Colab

Before touching the ESP32, we verified that the generated INT8 model could actually be loaded by TensorFlow Lite.

The interpreter reported:

Input

Shape: [1, 1]
dtype: int8

Input quantization:

scale = 0.012319971807301044
zero_point = 127

Output

Shape: [1, 1]
dtype: int8

Output quantization:

scale = 0.0022143314126878977
zero_point = 127

So we confirmed that this was genuinely an **INT8 TFLite model**, rather than merely a float model renamed or wrapped differently.

---

8. Local Numerical Test

We also tested the model against several values.

For example:

x = -3.142

model ≈ -0.0531
sin(x) ≈ 0
x = 0
model = 0
sin(x) = 0
x = 3.142
model ≈ -0.0089
sin(x) ≈ 0

However, some values had large errors.

For example:

x = 1
model ≈ -0.5647
sin(1) ≈ 0.8415

So the model's numerical accuracy was poor.

Important interpretation

That **does not invalidate this experiment**.

The purpose at this stage was:

> **Can the ESP32 execute a TFLite INT8 neural network?**

not:

> Can this particular tiny network approximate sine accurately?

The model is essentially a **compatibility payload**.

---

9. ESP-IDF Installation

We then moved to the physical deployment environment.

We selected:

ESP-IDF 5.5.5

The Espressif Installation Manager was installed and used to configure the ESP-IDF environment.

A major requirement was:

> Keep the ESP-IDF installation on the D: drive rather than C:.

The final important paths were:

IDF_PATH
D:\HAVELLS-PROJ\SLM\.espressif\v5.5.5\esp-idf
IDF_TOOLS_PATH
D:\HAVELLS-PROJ\SLM\.espressif\v5.5.5\tools
IDF_PYTHON_ENV_PATH
D:\HAVELLS-PROJ\SLM\.espressif\v5.5.5\tools\python\v5.5.5\venv

We verified:

ESP-IDF v5.5.5

was working.

---

10. ESP-IDF Project Creation

Inside:

D:\HAVELLS-PROJ\SLM

we created:

esp32_tflite_test

using:

idf.py create-project esp32_tflite_test

The resulting project was:

D:\HAVELLS-PROJ\SLM\esp32_tflite_test

---

11. ESP32 Target Configuration

Inside the project we executed:

idf.py set-target esp32

This successfully configured the project for the **original ESP32**, rather than ESP32-S3, C3, etc.

---

12. Baseline ESP-IDF Build

Before introducing TFLite, we built the vanilla project.

The baseline application was approximately:

0x27880 bytes

and the 1 MB application partition had approximately:

85% free

This was important because it established a **baseline firmware footprint** before adding the TFLite Micro stack.

---

13. Initial Flash Test

We then successfully flashed the ESP-IDF application onto the physical board.

There was one initial issue:

PermissionError(13): Access is denied

This was caused by the serial port being occupied/locked.

We closed Arduino IDE and reconnected the ESP32.

After that, flashing through:

COM6

worked successfully.

---

14. Flash Configuration Correction

Initially, ESP-IDF's boot output showed:

SPI Flash Size : 2MB

even though the chip detection showed 4 MB.

We therefore checked:

idf.py menuconfig

and inspected:

Serial flasher config

The setting was already configured as:

Flash size: 4 MB

After rebuilding, the flashing command explicitly used:

--flash_size 4MB

Later boot logs confirmed:

SPI Flash Size : 4MB

So the flash configuration is now correctly established.

---

15. Installing TFLite Micro

We added Espressif's TensorFlow Lite Micro component using:

idf.py add-dependency "esp-tflite-micro"

This successfully brought in:

espressif/esp-tflite-micro 1.4.0

and its dependency:

espressif/esp-nn 1.3.2

So the embedded inference stack became:

```
ESP-IDF 5.5.5
      │
      ├── esp-tflite-micro 1.4.0
      │
      └── esp-nn 1.3.2
```

---

16. Bringing the Model Onto Windows

The INT8 model was downloaded from Colab to Windows.

It was placed inside:

main/model/

specifically:

main/model/tiny_sine_int8.tflite

We verified its presence and size:

Exists: True
Size: 3344 bytes

---

17. Converting the `.tflite` Model Into C++

Microcontrollers generally don't load arbitrary desktop filesystem files in the same way a PC does.

Instead, we embedded the model directly into the firmware.

We therefore converted:

tiny_sine_int8.tflite

into:

tiny_sine_int8_model.cc

The generated C++ source contained:

```
const unsigned char tiny_sine_int8_model[] = {
    ...
};
```

and:

```
const unsigned int tiny_sine_int8_model_len = 3344;
```

The generated source itself was around:

16.8 KB

because binary data represented as hexadecimal C/C++ literals takes significantly more source-text space than the original binary.

---

18. CMake Integration

We modified:

main/CMakeLists.txt

so the ESP-IDF build included both:

esp32_tflite_test.cpp

and:

model/tiny_sine_int8_model.cc

with the model directory available as an include directory.

The final structure is essentially:

```
esp32_tflite_test/
│
├── main/
│   ├── CMakeLists.txt
│   ├── esp32_tflite_test.cpp
│   │
│   └── model/
│       ├── tiny_sine_int8_model.cc
│       └── tiny_sine_int8_model.h
│
├── managed_components/
│   ├── esp-tflite-micro
│   └── esp-nn
│
└── build/
```

---

19. First C/C++ Issue

Our first TFLite application source was:

esp32_tflite_test.c

But TFLite Micro's `MicroInterpreter` API is C++.

The compiler therefore produced:

fatal error: cstddef: No such file or directory

The underlying issue was:

C source
  ↓
C compiler
  ↓
C++ TFLite Micro headers
  ↓
failure

We corrected this by changing:

esp32_tflite_test.c

to:

esp32_tflite_test.cpp

and updating CMake accordingly.

After that, the compiler detected both:

C compiler
CXX compiler

and the TFLite code could be compiled as C++.

---

20. Second C++ Linker Issue

The next problem was:

undefined reference to `tiny_sine_int8_model'

The header declared the model externally, but the C++ model definition had linkage behavior that prevented the linker from resolving the symbol correctly.

We corrected the model definition to use C linkage:

extern "C"

and matched the declaration in the header.

After this change, the linker successfully connected:

esp32_tflite_test.cpp

↓
tiny_sine_int8_model.h
↓
tiny_sine_int8_model.cc
↓
embedded model

---

21. Successful TFLite Micro Build

After fixing the C++/linkage issues, the build succeeded.

The resulting application binary was:

0x31420 bytes

which is approximately:

201.8 KB

The 1 MB application partition had:

81% free

So the addition of TFLite Micro increased the firmware footprint substantially compared with the baseline.

That's expected because the firmware now contains:

- TFLite Micro runtime
- TensorFlow Lite operators
- FlatBuffers/model schema handling
- ESP-NN
- embedded model
- tensor/interpreter code

---

22. Flashing the TFLite Firmware

We flashed the new firmware:

idf.py -p COM6 flash

The ESP32 was detected as:

ESP32-D0WD-V3

revision v3.1

The application:

201760 bytes

was written to flash.

The important part:

Hash of data verified.

So the firmware was not merely transmitted — the written data was verified.

The ESP32 was then hard-reset.

---

23. Actual TFLite Micro Initialization

This is where the experiment became significantly more meaningful.

We added actual TFLite Micro initialization:

Model loading

```
const tflite::Model *model =
    tflite::GetModel(tiny_sine_int8_model);
```

Operator registration

We registered:

FullyConnected
ReLU

because those are the operations used by our tiny neural network.

Tensor arena

We allocated:

20 KB

for TFLite Micro's working memory:

constexpr int kTensorArenaSize = 20 * 1024;

Interpreter

We created:

MicroInterpreter

and then called:

interpreter.AllocateTensors();

---

24. Actual Hardware Result

The physical ESP32 produced:

TFLITE_TEST: ESP32 TFLite Micro test starting.

Then:

TFLITE_TEST: Model loaded successfully.

Then:

TFLITE_TEST: Model schema version: 3

Then:

TFLITE_TEST: AllocateTensors() succeeded.

And finally:

TFLITE_TEST: TFLite Micro model is running on ESP32!

This is the most important result of the entire experiment so far.

---

25. What We Have Definitively Proven

At this point, we can confidently say:

Hardware

ESP32-D0WD-V3

works with our deployment environment.

Software

ESP-IDF 5.5.5

works.

Embedded ML runtime

esp-tflite-micro 1.4.0

works on this setup.

Optimization library

esp-nn 1.3.2

is integrated.

Model

The:

3344-byte INT8 TFLite model

can be embedded into the firmware.

Model parser

TFLite Micro successfully parses it:

Model loaded successfully.

Tensor allocation

The runtime successfully allocates its tensors:

AllocateTensors() succeeded.

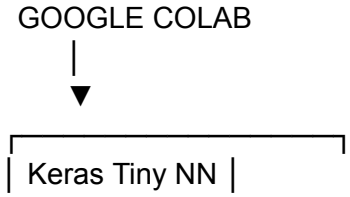
Physical execution environment

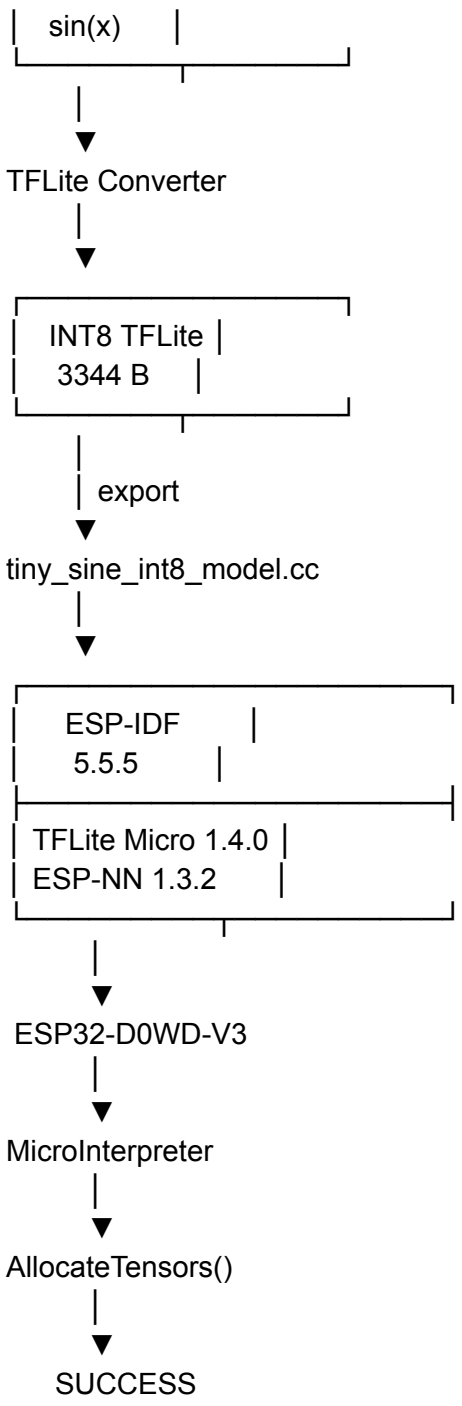
The model is now recognized by TFLite Micro **on the physical ESP32**, not just on the PC.

---

26. Current Architecture

Our current experimental pipeline is:




sin(x)
TFLite Converter
INT8 TFLite
3344 B
export
tiny_sine_int8_model.cc
ESP-IDF
5.5.5
TFLite Micro 1.4.0
ESP-NN 1.3.2
ESP32-D0WD-V3
MicroInterpreter
AllocateTensors()
SUCCESS

main/esp32_tflite_test.cpp (earlier)

```
#include <stdio.h>
#include "esp_log.h"

#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"

#include "tiny_sine_int8_model.h"

static const char *TAG = "TFLITE_TEST";

// Start small; we'll increase this only if required.
constexpr int kTensorArenaSize = 20 * 1024;

static uint8_t tensor_arena[kTensorArenaSize];

extern "C" void app_main(void)
{
   ESP_LOGI(TAG, "ESP32 TFLite Micro test starting...");

   // Load the embedded TFLite model.
   const tflite::Model *model =
      tflite::GetModel(tiny_sine_int8_model);

   if (model == nullptr) {
      ESP_LOGE(TAG, "Failed to load model!");
      return;
   }

   ESP_LOGI(TAG, "Model loaded successfully.");

   if (model->version() != TFLITE_SCHEMA_VERSION) {
      ESP_LOGE(TAG,
            "Model schema version %d does not match supported version %d",
            model->version(),
            TFLITE_SCHEMA_VERSION);
      return;
   }

   ESP_LOGI(TAG, "Model schema version: %d", model->version());

   // Register the operators required by our tiny model.
   static tflite::MicroMutableOpResolver<4> resolver;

   resolver.AddFullyConnected();
   resolver.AddRelu();
```

```
    // Create the interpreter.
    static tflite::MicroInterpreter interpreter(
        model,
        resolver,
        tensor_arena,
        kTensorArenaSize);

    // Allocate tensors.
    TfLiteStatus status = interpreter.AllocateTensors();

    if (status != kTfLiteOk) {
        ESP_LOGE(TAG, "AllocateTensors() failed!");
        return;
    }

    ESP_LOGI(TAG, "AllocateTensors() succeeded.");
    ESP_LOGI(TAG, "TFLite Micro model is running on ESP32!");
}
```

main/esp32_tflite_test.cpp

```
#include <stdio.h>
#include "esp_log.h"
#include "esp_timer.h"

#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"

#include "tiny_sine_int8_model.h"

static const char *TAG = "TFLITE_TEST";

constexpr int kTensorArenaSize = 20 * 1024;
static uint8_t tensor_arena[kTensorArenaSize];

extern "C" void app_main(void)
{
    ESP_LOGI(TAG, "ESP32 TFLite Micro latency test starting...");

    const tflite::Model *model =
        tflite::GetModel(tiny_sine_int8_model);

    if (model == nullptr) {
        ESP_LOGE(TAG, "Failed to load model!");
        return;
    }

    ESP_LOGI(TAG, "Model loaded successfully.");

    if (model->version() != TFLITE_SCHEMA_VERSION) {
        ESP_LOGE(TAG, "Model schema mismatch.");
        return;
    }

    static tflite::MicroMutableOpResolver<4> resolver;

    resolver.AddFullyConnected();
    resolver.AddRelu();

    static tflite::MicroInterpreter interpreter(
        model,
        resolver,
        tensor_arena,
        kTensorArenaSize);

    if (interpreter.AllocateTensors() != kTfLiteOk) {
```

```
    ESP_LOGE(TAG, "AllocateTensors() failed!");
    return;
}

ESP_LOGI(TAG, "AllocateTensors() succeeded.");

TfLiteTensor *input = interpreter.input(0);
TfLiteTensor *output = interpreter.output(0);

ESP_LOGI(TAG,
        "Input: type=%d scale=%f zero_point=%d",
        input->type,
        input->params.scale,
        input->params.zero_point);

ESP_LOGI(TAG,
        "Output: type=%d scale=%f zero_point=%d",
        output->type,
        output->params.scale,
        output->params.zero_point);

// ---------------------------------------------------------
// Warm-up inference
// ---------------------------------------------------------

float x = 0.0f;

int32_t quantized =
    (int32_t)(x / input->params.scale)
    + input->params.zero_point;

if (quantized > 127) quantized = 127;
if (quantized < -128) quantized = -128;

input->data.int8[0] = (int8_t)quantized;

if (interpreter.Invoke() != kTfLiteOk) {
    ESP_LOGE(TAG, "Warm-up Invoke() failed!");
    return;
}

ESP_LOGI(TAG, "Warm-up inference successful.");

// ---------------------------------------------------------
// Latency measurement
// ---------------------------------------------------------

constexpr int kRuns = 100;
```

```
int64_t total_us = 0;
int64_t min_us = INT64_MAX;
int64_t max_us = 0;

for (int i = 0; i < kRuns; i++) {

    int64_t start = esp_timer_get_time();

    TfLiteStatus status = interpreter.Invoke();

    int64_t end = esp_timer_get_time();

    if (status != kTfLiteOk) {
        ESP_LOGE(TAG, "Invoke() failed at run %d!", i);
        return;
    }

    int64_t latency_us = end - start;

    total_us += latency_us;

    if (latency_us < min_us) {
        min_us = latency_us;
    }

    if (latency_us > max_us) {
        max_us = latency_us;
    }
}

float average_us =
    (float)total_us / (float)kRuns;

// ---------------------------------------------------------
// Final result
// ---------------------------------------------------------

float result =
    (output->data.int8[0] - output->params.zero_point)
    * output->params.scale;

ESP_LOGI(TAG, "----------------------------------------");
ESP_LOGI(TAG, "Inference benchmark complete");
ESP_LOGI(TAG, "Runs       : %d", kRuns);
ESP_LOGI(TAG, "Min latency: %lld us", min_us);
ESP_LOGI(TAG, "Max latency: %lld us", max_us);
ESP_LOGI(TAG, "Avg latency: %.2f us", average_us);
```

```
    ESP_LOGI(TAG, "Avg latency: %.3f ms", average_us / 1000.0f);
    ESP_LOGI(TAG, "Final output: %f", result);
    ESP_LOGI(TAG, "----------------------------------------");
}
```

```
I (278) spi_flash: detected chip: generic                                    I
(282) spi_flash: flash io: dio                                          I (286)
main_task: Started on CPU0                                              I (286)
main_task: Calling app_main()                                           I (286)
TFLITE_TEST: ESP32 TFLite Micro latency test starting...
I (296) TFLITE_TEST: Model loaded successfully.
I (296) TFLITE_TEST: AllocateTensors() succeeded.
I (306) TFLITE_TEST: Input: type=9 scale=0.012320 zero_point=127
I (306) TFLITE_TEST: Output: type=9 scale=0.002214 zero_point=127
I (316) TFLITE_TEST: Warm-up inference successful.
I (326) TFLITE_TEST: ----------------------------------------              I
(326) TFLITE_TEST: Inference benchmark complete
I (326) TFLITE_TEST: Runs       : 100                                       I
(326) TFLITE_TEST: Min latency: 88 us                                        I
(336) TFLITE_TEST: Max latency: 129 us                                         I
(336) TFLITE_TEST: Avg latency: 89.20 us                                       I
(346) TFLITE_TEST: Avg latency: 0.089 ms
I (346) TFLITE_TEST: Final output: -0.050930
I (346) TFLITE_TEST: ----------------------------------------              I
(356) main_task: Returned from app_main()
```

What was our actual goal?

The broader project is your **SLM / TinyML Washing Machine project**.

The eventual system is:

```
User command / NLP input
        ↓
      SLM
        ↓
 Structured washing-machine action
        ↓
 Safety + Validation Layer
        ↓
 Deterministic Hardware Controller
        ↓
 ESP32
```

↓
Washing-machine hardware

But **we deliberately did NOT start by trying to cram SmolLM2-135M onto the ESP32.**

Our first objective was to establish whether the **deployment pipeline itself works on the original ESP32 hardware**.

🧪 Our current experiment

We created a deliberately tiny neural network:

Input
  |
  ▼
Dense(16, ReLU)
  |
  ▼
Dense(16, ReLU)
  |
  ▼
Dense(1)

trained it in Colab, converted it to **INT8 TFLite**, embedded the `.tflite` binary into the ESP-IDF project, and deployed it using **TFLite Micro**.

Hardware:

DOIT ESP32 DEVKIT V1
ESP32-D0WD-V3
4 MB Flash

✅ What we've proven so far

1. TFLite Micro compatibility

The ESP32 successfully:

Loaded model
↓
Checked schema
↓
Created MicroInterpreter

↓
Allocated tensors

PASS

---

2. Actual neural-network inference

We didn't just load the model.

We executed:

interpreter.Invoke()

on the physical ESP32.

PASS

---

3. INT8 inference

The hardware reported:

Input:
type = 9
scale = 0.012320
zero_point = 127

Output:
type = 9
scale = 0.002214
zero_point = 127

`type=9` corresponds to **INT8**.

So we're genuinely testing an INT8 quantized model.

PASS

---

4. Latency

We ran **100 inferences** after a warm-up.

Minimum = 88 µs
Maximum = 129 µs
Average = 89.20 µs

= 0.0892 ms

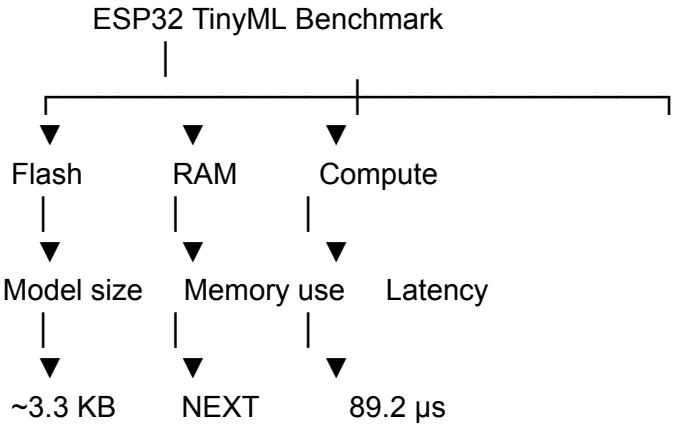
So:

~89 µs average inference latency on the ESP32.

PASS

📊 What remains in this compatibility benchmark?

We're essentially building a **hardware deployment profile**:



We already have the model/firmware Flash numbers from the build:

Firmware ≈ 203 KB
App partition remaining ≈ 81%

But **that isn't the same thing as the model's RAM requirement**.

That's why we're moving to the next measurement:

🔥 RAM footprint

We want to determine how much memory TFLite Micro consumes when we go from:

ESP32
↓
TFLite runtime
↓
Model loaded

↓
Tensor arena allocated
↓
Inference

We'll measure the ESP32 heap at those stages.

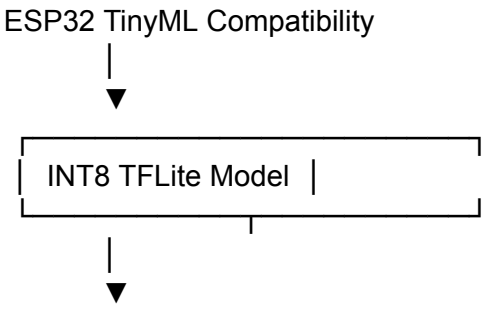
Eventually our benchmark should look something like:

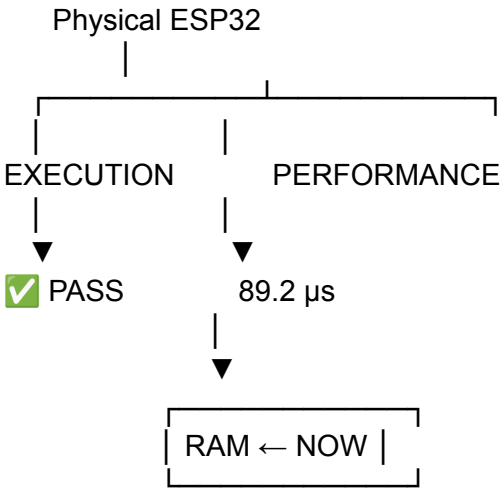
Metric	Result
Model format	TFLite INT8
Model size	**3,344 B**
Firmware size	**203,184 B**
Tensor arena	**20 KB allocated**
Actual heap impact	**NEXT**
Inference min	**88 µs**
Inference avg	**89.2 µs**
Inference max	**129 µs**
INT8	✅
Physical ESP32 execution	✅

And **only after we establish this baseline** do we move toward the actual washing-machine model.

🚀 So right now

We are **here**:





Next: RAM measurement.

We'll modify the current firmware minimally, build → flash → monitor, exactly like we did for latency. No jumping ahead.

Our benchmark is coming together

We now have:

Metric	Measured result
Model	Tiny TFLite INT8
`.tflite` size	**3,344 B**
Firmware	**203,040 B**
Tensor arena reserved	**20 KB**
Dynamic heap change after allocation	**184 B**
Dynamic heap change after inference	**412 B**
Inference runs	**100**
Min latency	**88 µs**
Avg latency	**89.2 µs**
Max latency	**129 µs**

Physical ESP32 inference	✅
INT8 execution	✅

This is already a legitimate **ESP32 TinyML deployment baseline**.

🚀 What should we measure next?

Before putting the real washing-machine classifier/parser on the board, there's one particularly useful metric left:

Flash footprint

We already know the overall firmware is ~203 KB, but we should explicitly separate:

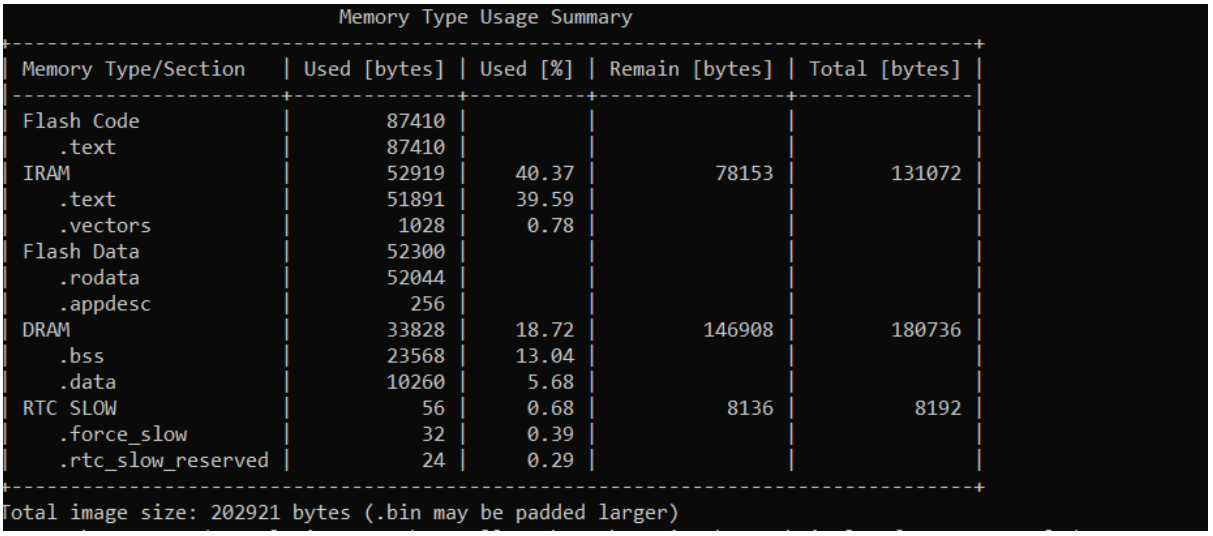
```
4 MB ESP32 Flash
│
├── Bootloader
├── Partition table
├── Application
│     ├── TFLite Micro runtime
│     ├── ESP-NN
│     ├── Model
│     └── Application code
└── Remaining space
```

That gives us a proper **Flash + RAM + latency profile**.

Then we can compare the eventual washing-machine model against this baseline and answer the important question:

> **Can the actual SLM/TinyML model fit and execute within the ESP32's resource budget?**

We're progressing exactly in the right order: **prove → measure → optimize → replace the toy model with the real task model.**



Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	87410			
.text	87410			
IRAM	52919	40.37	78153	131072
.text	51891	39.59		
.vectors	1028	0.78		
Flash Data	52300			
.rodata	52044			
.appdesc	256			
DRAM	33828	18.72	146908	180736
.bss	23568	13.04		
.data	10260	5.68		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Total image size: 202921 bytes (.bin may be padded larger)

Date: 16 Sep 2026
Current milestone: PC reference generation VERIFIED
Next milestone: Port genuine autoregressive SLM inference to physical ESP32-D0WD

---

1. Ultimate Objective

The goal is **actual language-model inference on a physical ESP32**, eventually for the washing-machine command system.

Target architecture:

Natural-language command
↓
Tiny SLM / LLM
↓
Structured washing-machine command
↓
Safety + Validation Layer
↓
Deterministic Controller
↓
ESP32
↓
Washing-machine hardware

Important: the earlier TFLite experiment was a **deployment smoke test**, not the final model.

---

2. Hardware

Board

DOIT ESP32 DEVKIT V1
ESP32-D0WD-V3
Revision: v3.1
Flash: 4 MB
PSRAM: None assumed
Port: COM6

Current ESP-IDF firmware previously reported CPU at 160 MHz. **Do not change this yet.**

---

3. Existing TinyML Baseline — COMPLETE

Separate project:

D:\HAVELLS-PROJ\SLM\esp32_tflite_test

Runtime:

ESP-IDF 5.5.5
espressif/esp-tflite-micro 1.4.0
esp-nn 1.3.2

Model:

Tiny Dense NN
INT8 TFLite
Size: 3344 bytes

Physical ESP32 inference:

100 runs
Min: 88 µs
Avg: 89.2 µs
Max: 129 µs

Memory:

Tensor arena: 20 KB reserved
Dynamic heap Δ after AllocateTensors: 184 B
Dynamic heap Δ after inference: 412 B

Firmware:

Total image: ~202,921 B
App partition free: 81%

Status

Physical INT8 TFLite inference: PASS

This project should remain untouched as the deployment baseline.

---

4. Existing SLM Quality Benchmark — DO NOT OVERWRITE

Separate research/model work:

SmolLM2-135M-Instruct
+
repaired-label LoRA

Frozen benchmark:

41 / 45 exact
= 91.11%

This remains the **quality/reference benchmark** for the eventual washing-machine command model.

It is **not** the immediate ESP32-D0WD deployment target.

---

5. Actual ESP32 SLM Experiment

New project:

D:\HAVELLS-PROJ\SLM\esp32_llm_test

Current tree:

```
esp32_llm_test
│
├── model
│   ├── stories260K.bin
```

```
│   └── tok512.bin
│
└── reference
    └── llama2.c
```

Downloaded model sizes:

```
stories260K.bin = 1,056,540 bytes
tok512.bin      = 6,227 bytes
```

Approximately:

```
Model     ≈ 1.01 MiB
Tokenizer ≈ 6.08 KiB
```

---

6. Why TinyStories 260K?

We deliberately chose a **genuine autoregressive transformer**, rather than another classifier.

Checkpoint header decoded as:

```
dim        = 64
hidden_dim = 172
n_layers   = 5
n_heads    = 8
n_kv_heads = 4
vocab_size = 512
```

This is small enough to investigate on the original ESP32-D0WD while still being an actual language model.

The purpose is to establish:

```
model loading
tokenization
transformer inference
autoregressive generation
RAM usage
Flash usage
latency/token
tokens/sec
context limits
```

before attempting a larger/task-specific SLM.

---

7. Reference Runtime

Repository cloned into:

D:\HAVELLS-PROJ\SLM\esp32_llm_test\reference

Repository:

karpathy/llama2.c

Important files:

run.c
runq.c
win.c
win.h
sample.py
tokenizer.py
model.py
export.py

The repository provides:

build_msvc.bat

which contains:

cl.exe /fp:fast /Ox /openmp /I. run.c win.c

---

8. Windows Toolchain Issue — RESOLVED

Initial compiler:

MinGW.org GCC 6.3.0

It failed because of compatibility/linker issues involving:

_ftelli64
mmap
munmap
clock_gettime

We installed:

Visual Studio Build Tools 2022

and added:

Desktop development with C++

MSVC confirmed:

Microsoft (R) C/C++ Optimizing Compiler
Version 19.44.35228

x64 environment activated using:

call "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\vcvars64.bat"

Confirmed:

Environment initialized for: 'x64'

---

9. PC Reference Runtime — VERIFIED

Built:

cl.exe /fp:fast /O2 /openmp /I. run.c win.c

Successful:

run.exe
Size: 228,352 bytes
Target: x64

---

10. Important Failed Test

Initially we ran:

run.exe stories260K.bin -n 50 -s 42 -i "Once upon a time"

It produced no output.

Exit code:

-1073741819

which corresponds to Windows:

0xC0000005
ACCESS VIOLATION

We investigated x86 vs x64.

The x64 build produced the same crash.

Actual cause discovered

The TinyStories 260K checkpoint requires its **custom 512-token tokenizer**.

We had omitted:

-z tok512.bin

The runtime therefore used its default tokenizer, which was incorrect for this checkpoint.

---

11. GOLDEN REFERENCE — VERIFIED ✅

Correct command:

run.exe D:\HAVELLS-PROJ\SLM\esp32_llm_test\model\stories260K.bin -z D:\HAVELLS-PROJ\SLM\esp32_llm_test\model\tok512.bin -t 0.0 -n 50

Successful generation:

Once upon a time, there was a little girl named Lily. She loved to play outside in the park. One day, she saw a big, red ball. She wanted to play with

Measured PC speed:

3266.666667 tok/s

This is our GOLDEN REFERENCE.

The exact combination is:

stories260K.bin
+
tok512.bin
+
llama2.c run.c/win.c
↓
VALID GENERATION

Do not change the checkpoint or tokenizer.

---

12. Current Exact Position

We are **DONE with PC validation**.

We have proven:

```
stories260K.bin
      +
   tok512
      ↓
llama2.c reference
      ↓
  PC inference
      ↓
GENERATED TEXT ✅
```

The next work is **ESP32 porting**.

---

13. NEXT SESSION — DO THIS FIRST

Do NOT immediately start coding ESP32 inference.

First inspect the reference implementation and identify the minimum pieces required.

Specifically inspect `run.c` for:

1. checkpoint loading
2. Config structure
3. TransformerWeights
4. RunState
5. tokenizer
6. transformer forward()
7. sampling
8. main()
9. mmap / munmap
10. timing functions
11. malloc/free

We need to separate:

DESKTOP-ONLY
↓
file mapping
Windows/POSIX timing
CLI parsing
stdout
etc.

from:

MODEL CORE
↓
checkpoint format
tokenization
embedding
RMSNorm
RoPE
attention
KV cache
FFN
logits
sampling

Then create a **minimal ESP-IDF implementation** around the same checkpoint format.

---

14. Critical Constraint

The target is **original ESP32-D0WD**, not ESP32-S3.

Do not assume results from:

ESP32-S3
ESP32-C3
boards with PSRAM

apply directly to this board.

Our actual question is:

> **Can a genuine ~260K-parameter autoregressive language model perform token generation on an ESP32-D0WD-V3 with 4 MB Flash and no PSRAM?**

We will measure it rather than assume.

---

15. Experimental Roadmap

